- *Implicit file includes*—Previously, every file in the project had to be listed in the .csproj file to be included in the build. Now files are compiled automatically.
- *No paths to NuGet package .dll files*—Previously, you had to include the path to the .dll files contained in NuGet packages in the .csproj, as well as list the dependencies in a packages.config file. Now you can reference the NuGet package directly in your .csproj, and you don't need to specify the path on disk.

All these changes combine to make the project file far more compact than you'll be used to from previous .NET projects. The following listing shows the entire .csproj file for your sample app.

Listing 3.2 The .csproj project file, showing SDK, target framework, and references

```
<Project Sdk="Microsoft.NET.Sdk.Web">                ❶
  <PropertyGroup>
    <TargetFramework>net7.0</TargetFramework>        ❷
    <Nullable>enable</Nullable>                      ❸
    <ImplicitUsings>enable</ImplicitUsings>          ❹
  </PropertyGroup>
</Project>
```

❶ The SDK attribute specifies the type of project you're building.

❷ The TargetFramework is the framework you'll run on—in this case, .NET 7.

❸ Enables the C# 8 feature "nullable reference types"

❹ Enables the C# 10 feature "implicit using statements"

For simple applications, you probably won't need to change the project file much. The `Sdk` attribute on the `Project` element includes default settings that describe how to build your project, whereas the `TargetFramework` element describes the framework your application will run on. For .NET 6.0 projects, this element will have the `net6.0` value; if you're running on .NET 7, this will be `net7.0`. You can also enable and disable various features of the compiler, such